AlgoBench

Algorithm Benchmarking & Profiling Suite

SSL Project Team

Department of Computer Science and Engineering
Indian Institute of Technology Dharwad

November 2026

CS25BT005 MC25BT022 CS25BT031 CS25BT041 CS25BT051

Problem and Motivation

- Complexity analysis explains growth trends, but not every practical implementation difference.
- Observed performance depends on input size, input distribution, implementation, and memory allocation.
- Repeated experiments are needed to compare algorithms under controlled, reproducible conditions.

Our goal

Build one environment to configure experiments, measure algorithms, inspect bottlenecks, and compare results.

What AlgoBench Does

1. Select a domain and algorithms.
2. Generate deterministic input datasets from a chosen seed.
3. Execute warm-up and repeated timing trials.
4. Measure peak additional Python memory separately.
5. Optionally inspect function costs with cProfile/pstats.
6. Save CSV results and visualize scaling with charts/GNUplot.

Five Algorithm Domains

Domain	Implementations
Sorting	Quick, Merge, Heap, Insertion Sort
Searching	Linear, Binary, Hash Table Lookup
Graph	Dijkstra, Bellman–Ford, BFS
Dynamic Programming	LCS and 0/1 Knapsack (memoization/tabulation)
Matrix	Standard and Strassen Multiplication

Total

16 registered algorithm implementations.

System Architecture

1. Streamlit Dashboard

Configure experiments, launch runs, compare results, inspect profiles, browse history.

⇓

2. Algorithm Registry + Dataset Generators

Resolve the selected callable and prepare its reproducible input arguments.

⇓

3. Generic Benchmark Engine

Repeated timing, separate memory measurement, optional cProfile, CSV persistence.

⇓

4. Visualization and History

Dashboard comparison charts, GNUplot runtime/memory plots, saved experiment history.

Measurement Methodology

- Runtime: `time.perf_counter()`, warm-up, repeated trials.
- Statistics: average, median, min, max, standard deviation.
- Memory: separate `tracemalloc` run; peak additional traced memory in KiB.
- Profiling: separate `cProfile`/`pstats` run; timing is not contaminated by profiler overhead.
- Reproducibility: dataset generators accept a random seed.

Fair comparison

Dataset preparation and input copying are excluded from the timed algorithm call and documented in the result notes.

Testing and Validation

- Algorithm correctness and input-validation tests.
- Reproducibility tests for graph, string, and matrix generators.
- Benchmark timer, memory, profiler, CSV, and GNUplot data tests.
- Integration smoke tests execute all 16 registered algorithms through the common engine.

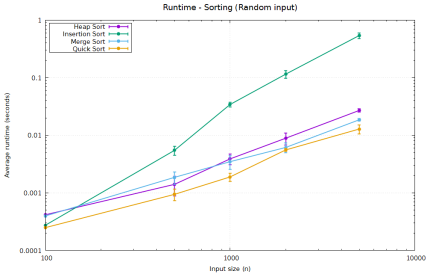
Integration test result

158 pytest cases passed, plus 64 subtests, in the integration environment.

Streamlit was run locally; all five domains completed and GNUplot rendered runtime and memory plots.

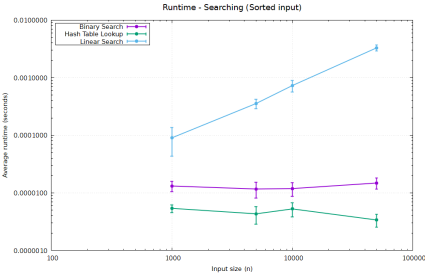
Measured Results: Sorting and Searching

Sorting: runtime



Random arrays, $n = 100$ to $5,000$; five trials per point.

Searching: runtime



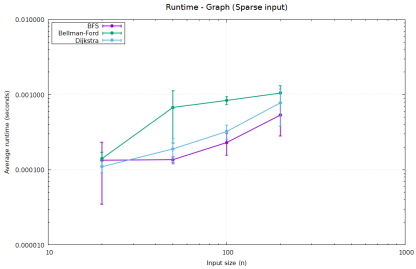
Successful search on sorted arrays, $n = 1,000$ to $50,000$.

Observed at largest tested size

At $n = 5,000$, Insertion Sort took 0.540 s versus Quick Sort's 0.0128 s. At $n = 50,000$, Linear Search took 3.296 ms; Binary Search took 0.0148 ms.

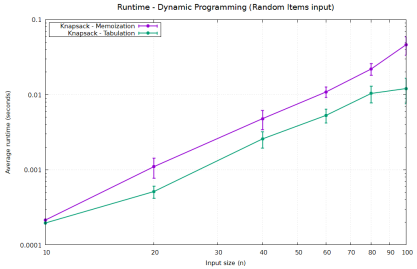
Measured Results: Graph and Dynamic Programming

Sparse graph runtime



20–200 vertices.

Knapsack runtime



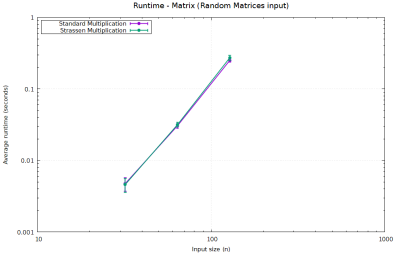
10–100 items.

Observed at largest tested size

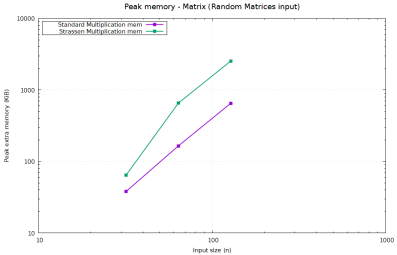
At 200 vertices, Dijkstra took 0.781 ms and Bellman–Ford 1.051 ms. At 100 items, Knapsack tabulation took 12.086 ms versus 46.322 ms for memoization.

Measured Results: Matrix Multiplication

Runtime



Peak additional memory



Dimension 128

Standard: 249.948 ms and 650.7 KiB. Strassen: 272.290 ms and 2,533.3 KiB. The tested sizes did not reach a Strassen runtime crossover.

Profiler Findings and Interpretation

- Quick Sort ($n = 5,000$): 5,971 recursive `sort` calls in the profile.
- Knapsack memoization ($n = 100$): 45,463 recursive `solve` calls and 44,869 calls to `max`.
- Strassen ($n = 128$): 57 recursive calls, plus repeated matrix add/subtract and base-case multiplication.

Measurement caveat

Some cProfile captures also include Streamlit watcher and event-loop activity. Their cumulative times are not attributable solely to the algorithm; isolate profiling in a worker process in a future version.

Validation

158 pytest cases and 64 subtests passed in the integration test environment; all 16 registered algorithms were smoke-tested through the engine. The local dashboard generated five-category CSV and GNUplot evidence.

Five-Minute Demonstration

1. 0:00–0:40 — Problem and AlgoBench overview.
2. 0:40–1:30 — Select category, algorithms, input distribution, and sizes.
3. 1:30–2:30 — Run experiment; inspect runtime and memory results.
4. 2:30–3:20 — Compare scaling and show CSV/GNUplot output.
5. 3:20–4:15 — Show cProfile bottleneck report.
6. 4:15–5:00 — Show tests, Bash/Makefile automation, and conclusion.

Conclusion

- AlgoBench connects algorithm implementations to a reusable experiment engine.
- Reproducible datasets and repeated measurements support meaningful comparisons.
- Separate timing, memory, and profiling runs improve measurement clarity.
- CSV history and GNUplot provide reproducible evidence for analysis.

Thank you

Questions?